

Firestore como Backend en Vue 3

Autenticación, base de datos, reglas y servicios cloud integrados con Composition API.



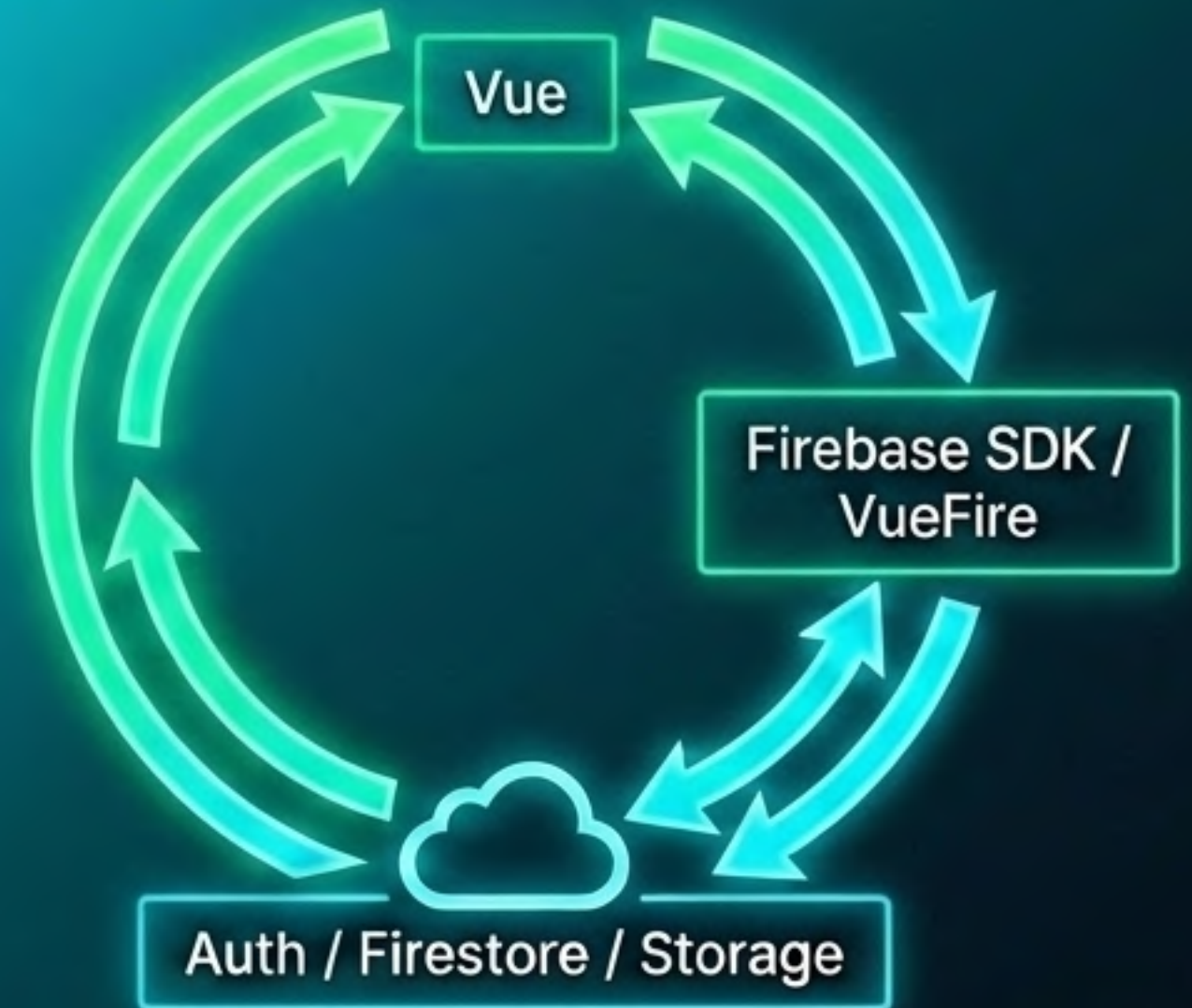
Del consumo de APIs a la sincronización en tiempo real

Consumo Tradicional de APIs



Requiere peticiones manuales, manejo explícito de carga (loading), y recargas forzadas al mutar datos.

Backend Administrado con VueFire



El estado reactivo de Vue puede sincronizarse en tiempo real con servicios como Firestore. No solo consumes datos, sincronizas el estado con la nube.

Anatomía de la capa invisible de tu aplicación

El backend es la infraestructura detrás de escena que asegura que tu aplicación sea persistente, segura y funcional para múltiples usuarios.



Arquitectura Tradicional vs. Backend as a Service (BaaS)

Servidor Tradicional (Node, Laravel, Django)	Backend as a Service (Firebase)
Infraestructura: Requiere configurar, parchear y mantener servidores virtuales y bases de datos. 	Entrega servicios en la nube llave en mano, administrados por Google Cloud. 
Velocidad inicial: Desarrollo inicial más lento; requiere crear la API desde cero. 	Desarrollo extremadamente rápido; los SDKs conectan la UI directamente a la nube. 
Escalabilidad: no manual (balanceadores de carga, réplicas de bases de datos). 	Escala automáticamente desde cero hasta millones de usuarios sin intervención. 
Control absoluto sobre el código subyacente y el motor de base de datos. 	Menos control de infraestructura bajo nivel; dependes del ecosistema y reglas de la plataforma. 

Firebase es una plataforma integral, no solo una base de datos

Nota: Muchas personas confunden
Firebase solo con su base de datos, pero
es un conjunto completo de herramientas.



Paso 1: Configuración e inicialización del proyecto

```
Terminal

→ # Crear app Vue e instalar dependencias principales
→ pnpm create vue@latest firebase-vue-app
→ cd firebase-vue-app
→ pnpm install

→ # Instalar SDK de Firebase y el puente VueFire
→ pnpm add firebase vuefire

→ # Iniciar servidor de desarrollo
→ pnpm dev
```

```
JS src/firebase.js ×

1  import { initializeApp } from 'firebase/app'
2  import { getFirestore } from 'firebase/firestore'
3
4  // El objeto de configuración te lo provee la consola de Firebase
5  const firebaseConfig = {
6    apiKey: '...',
7    authDomain: '...',
8    projectId: '...'
9  }
10 // Inicializamos e instanciamos la aplicación de Firebase
11 export const firebaseApp = initializeApp(firebaseConfig)
13 export const db = getFirestore(firebaseApp)|
```


Paso 2: Integrando VueFire con Vue



VueFire es la biblioteca oficial para integrar Firebase con Vue...

Provee composables que gestionan automáticamente las suscripciones a los **datos de la nube** sin tener que escribir código repetitivo de ciclo de vida.

JS src/main.js ×

```
1  import { createApp } from 'vue'
2  import { VueFire, VueFireAuth } from 'vuefire'
3  import App from './App.vue'
4  import { firebaseApp } from './firebase'
5
6  const app = createApp(App)
7
8  // Inyectar VueFire y el módulo de Autenticación
9  app.use(VueFire, {
10    firebaseApp,
11    modules: [
12      VueFireAuth()
13    ]
14  })
15
16  app.mount('#app')
```


Autenticación segura y sin fricciones



El servicio Authentication maneja inicios de sesión (email, Google, GitHub) y almacena las credenciales y tokens de sesión en el navegador automáticamente de forma segura.

En un caso real, email y password vienen desde inputs vinculados con v-model.

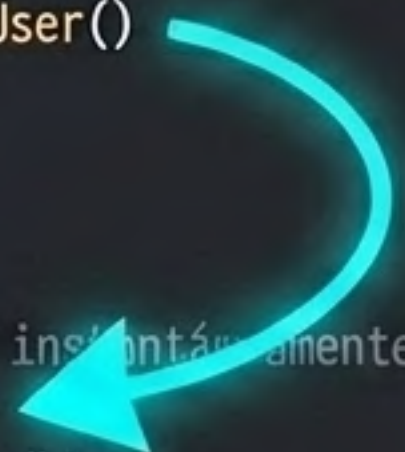
```
✓ Login.vue ×  
1 <script setup>  
2 import { ref } from 'vue'  
3 import { getAuth, signInWithEmailAndPassword } from 'firebase/auth'  
4  
5 const email = ref('')  
6 const password = ref('')  
7  
8 async function login() {  
9   try {  
10     const auth = getAuth()  
11     // Firebase valida y crea la sesión automáticamente  
12     await signInWithEmailAndPassword(auth, email.value, password.value)  
13     console.log('Inicio de sesión exitoso')  
14   } catch (error) {  
15     console.error('Credenciales inválidas', error)  
16   }  
17 }  
18 </script>
```

Idea Clave: Firebase maneja la persistencia de sesión por debajo. Tú solo llamas a la función y esperas el resultado.

Reaccionando al estado global de la sesión de usuario

El composable **useCurrentUser()** expone un Ref reactivo. Es null cuando no hay usuario autenticado, y expone los datos del usuario inmediatamente al loguearse.

```
✓ Perfil.vue x
1  <script setup>
2  import { useCurrentUser } from 'vuefire'
3
4  // Este ref reacciona automáticamente a logins, logouts o refrescos de
5  const user = useCurrentUser()
6  </script>
7
8  // Template
9  <template>
10   <!-- El DOM se adapta instantáneamente a la presencia del usuario -->
11   <section v-if='user'>
12     <h1>¡Hola de nuevo!</h1>
13     <p>Has iniciado sesión como: {{ user.email }}</p>
14     <p>Tu UID único es: {{ user.uid }}</p>
15   </section>
16
17   <section v-else>
18     <p>Acceso denegado. Por favor, inicia sesión para continuar.</p>
19   </section>
20 </template>
```



Modelando datos NoSQL en Firestore

Firestore es una base de datos en tiempo real, alojada en la nube y orientada a documentos, construida para escalar.

Colección



Un contenedor o agrupación lógica (similar a una tabla en SQL, aunque Firestore no funciona igual que una base relacional.)
Ejemplo: Una colección llamada products.

Documentos



Un registro individual dentro de la colección. Contiene pares clave-valor (campos), similar a un objeto JSON puro.
Ejemplo: Un producto individual con ID único, name, y price.

Rutas de acceso (Pathing)

```
/products/{productId}
/users/{userId}
/posts/{postId}/comments/{commentId}
```


Leyendo colecciones y sincronizando en tiempo real

El composable **useCollection** vincula directamente una colección de la nube con un array reactivo local. Si alguien añade un documento desde otro dispositivo, la lista de tu interfaz se actualiza sola.

ProductList.vue x

Sincronización en Tiempo Real Activa

```
1 <script setup>
2 import { collection } from 'firebase/firestore'
3 import { useCollection } from 'vuefire'
4 import { db } from '@/firebase' // Tu base de datos inicializada
5
6 // Array reactivo enlazado bidireccionalmente a la nube
7 const products = useCollection(collection(db, 'products'))
8 </script>
9
10
11 <template>
12   <ul>
13     <!-- Renderizado dinámico conforme Firestore cambia -->
14     <li v-for='product in products' :key='product.id'>
15       <strong>{{ product.name }}</strong> - ${{ product.price }}
16     </li>
17   </ul>
18 </template>
```


Guardando nuevos documentos en Firestore

VueFire simplifica la lectura reactiva de datos. Para crear, actualizar o borrar documentos usamos funciones del SDK de Firebase.

```
AddProduct.vue x
1  <script setup>
2  import { ref } from 'vue'
3  import { collection, addDoc, serverTimestamp } from 'firebase/firestore'
4  import { db } from '@firebase/database'
5
6  // Estado local para el formulario
7  const newName = ref('')
8  const newPrice = ref(0)
9
10 async function submitProduct() {
11   // Inserción en la colección 'products'. Firebase genera un ID
12   await addDoc(collection(db, 'products'), {
13     name: newName.value,
14     price: Number(newPrice.value),
15     createdAt: serverTimestamp()
16   })
17
18   // Limpiamos el formulario post-inserción
19   newName.value = ''
20   newPrice.value = 0
21 }
22 </script>
```

El useCollection detectará este cambio y actualizará la UI automáticamente.

La seguridad real debe validarse en el Backend



IDEA CLAVE: ¡Ocultar botones en Vue con `v-if='user'` NO es seguridad real! Un atacante experto puede obviar la interfaz y comunicarse directamente con tu base de datos mediante la API web. La protección infalible se debe declarar en el servidor.



 Firebase

Reglas de Seguridad de Firestore

```
// Sintaxis de validación que se aplica antes de tocar el disco
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    match /products/{productId} {
      // Público: Cualquier persona puede ver el catálogo
      allow read: if true;

      // Restringido: Solo usuarios con sesión válida pueden insertar/editar
      allow write: if request.auth != null;
    }
  }
}
```


Ecosistema expandido: Completando la arquitectura Full-Stack



Cloud Storage (Almacenamiento Binario)

Guarda archivos pesados que no caben en una base de datos (imágenes de perfil, PDFs, videos).

Sinergia: Vue sube la imagen a Storage; este devuelve una URL pública; Vue la guarda en Firestore.



Firebase Hosting (Despliegue)

Publica tu aplicación frontend en una CDN global rápida.

Comando clave: Configura rutas de SPAs y sube a producción corriendo **firebase deploy**.



Cloud Functions (Lógica Serverless)

Ejecuta código en servidores backend de forma oculta al usuario, detonado por eventos.

Ejemplo: Redimensionar imágenes subidas automáticamente, o integraciones de pagos seguras.

Arquitectura Completa: En Resumen



Firebase cubre gran parte del rol del backend: entrega infraestructura lista para producción sin provisionar máquinas virtuales.



Auth simplifica la identidad: Abstrae flujos complejos de inicio y fin de sesión y tokens de acceso.



Firestore guarda y sincroniza: Mantiene documentos NoSQL consistentes sin importar la escala.



VueFire es la capa de integración: Combina la Composition API de Vue con los datos en vivo, logrando una sincronización limpia (useCollection, currentUser).



Security Rules son la capa principal de protección de datos: Validan a nivel servidor la integridad y permisos, tapando los agujeros del frontend.

**Has conectado una app Vue reactiva con servicios backend en la nube.
¡Estás listo para construir aplicaciones full-stack modernas!**